Lab 4.3 (Optional): Building a Text Viewer

In this lab, you will implement a text viewer and control fonts for on-screen display.

To see a demonstration of the solution for this lab, click this icon.
(CD-ROM plays the demonstration, "Lab 4.3 Demonstration.")

Estimated time to complete this lab: **30 minutes**

To complete the exercises in this lab, you must have the required software. For detailed information about the labs and setup for the labs, see Labs in this course.

The solution code for this lab is located in the folder \Labs\Ch04\Lab4.3.

Objectives

After completing this lab, you will be able to:

- Implement supporting code for a viewer.
- Display streams of text.
- Control fonts for on-screen display.

Prerequisites

Before working on this lab, you should be familiar with the following:

- Member functions of **CDC** class

You may find it helpful to review the MSDN Library Visual Studio 98 Help topics on graphics before attempting this lab.

Exercises

The following exercises provide practice working with the concepts and techniques covered in this chapter:

Exercise 1: Implementing a Basic Text Viewer

In this exercise, you will add text display capability to a viewer.

Exercise 2: Adding Font Support

In this exercise, you will implement a user interface for the text viewer you created in Exercise 1.

123 Exercise 1: Implementing a Basic Text Viewer

In this exercise, you will add text display capability to a viewer based on **CScrollView**. There are four parts to this exercise:

1. Creating a multiple-document interface (MDI) application with MFC AppWizard
2. Adding file-handling to the document
3. Calculating the basic metrics of the view
4. Displaying the text

At the end of this exercise, you will have created an MDI text viewer with file selection, display, and scrolling.

▶ Use MFC AppWizard to create an MDI application

☐ 1. Name the new project workspace **Text**.

☐ 2. In Step 6, derive **CTextView** from **CScrollView,** rather than **CView**.

Finish and create the new project. Build it at this point.

In the Text.exe file, **CTextDoc** is little more than a holder for **CStringList**, into which you will read the lines of a selected file. **CDocument** provides the menu and the **File Open** dialog box. You will provide the file-handling code and the parsing of the file into **CStringList**.

▶ Prepare CTextDoc for file reading

☐ 1. Open the file TextDoc.h.

☐ 2. Declare a protected **CStringList** member:

```
CStringList m_LineList;
```

☐ 3. Declare a public member function to return a pointer to m_LineList:

```
CStringList*    GetLineList() { return &m_LineList; }
```

☐ 4. Save TextDoc.h.

☐ 5. **CTextDoc::OnNewDocument** is called when the application starts and when a user action occurs. Because Text.exe is a read-only application, disable this function in the file TextDoc.cpp:

```
BOOL CTextDoc::OnNewDocument()
{
    return FALSE;
}
```

Notes

▶ **Implement OnOpenDocument**

☐ 1. **CTextDoc::OnOpenDocument** will read the selected text file, line by line, into the **CStringList** member. **OnOpenDocument** is called from the application after **CTextApp** has queried the user with a **File Open** dialog box. Create **CTextDoc::OnOpenDocument** from ClassWizard or WizardBar. Edit the code to remove the default handler.

☐ 2. Reading a file into memory one line at a time could take a while. Show the wait cursor with **CCmdTarget::BeginWaitCursor**:

```
BeginWaitCursor();
```

☐ 3. Clear all the items from m_LineList:

```
m_LineList.RemoveAll();
```

☐ 4. **CStdioFile** provides stream-oriented file access with line-oriented file access. Open the file passed in lpszPathName by constructing a **CStdioFile** object as follows:

```
CStdioFile  file(lpszPathName,
                 CFile::modeRead | CFile::typeText);
```

☐ 5. Declare a **CString** variable into which to read each line as follows:

```
CString strLine;
```

☐ 6. **CStdioFile::ReadString** returns **TRUE** if anything was read and **FALSE** if the end of the file was encountered before reading any data. Read data as long as there is anything to read:

```
while (file.ReadString(strLine) != NULL)
{
```

☐ 7. Clean up the ends of the lines for white space and control characters as follows:

```
    int nLastCharIndex = strLine.GetLength()-1;
    while (nLastCharIndex >= 0 && strLine[nLastCharIndex] < ' ')
    {
        strLine.SetAt(nLastCharIndex--, '\0');
    }
```

☐ 8. Once the string is clean, add it to the end of m_LineList as follows:

```
m_LineList.AddTail(strLine);
```

☐ 9. At the end of the read loop, restore the cursor as follows:

```
EndWaitCursor();
```

☐ 10. Return **TRUE** to indicate that you have handled the message:

```
return TRUE;
```

☐ 11. Save TextDoc.cpp and TextDoc.h.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
BOOL CTextDoc::OnOpenDocument(LPCTSTR lpszPathName)
{
    //  Could be a big file
    BeginWaitCursor();

    //  Clear List, this will cleanup the CString objects
    m_LineList.RemoveAll();

    //  Read the file and store as a list
    //  of CStrings
    CStdioFile  file(lpszPathName,
                    CFile::modeRead | CFile::typeText);


    CString strLine;
    while (file.ReadString(strLine) != NULL)
    {
        //remove the noise characters at the end of the line
        int nLastCharIndex = strLine.GetLength()-1;
        while (nLastCharIndex >= 0 && strLine[nLastCharIndex] < ' ')
        {
        strLine.SetAt(nLastCharIndex--, '\0');
        }

        //  Add to CStringList
        m_LineList.AddTail(strLine);
    }

    EndWaitCursor();
    return TRUE;
}
```

▶ **Declare the basic metrics members**

You will need to have a number of basic metrics for the text view. These do not need to be calculated each time you draw text on the screen if you hold them in member variables.

☐ 1. Right-click **CTextView** in ClassView and add the following protected variables:

```
CSize   m_ViewCharSize
CSize   m_DocSize
CFont*  m_pFont
```

☐ 2. Right-click **CTextView** in ClassView and declare a public function as follows:

```
CFont*    GetFont()
```

☐ 3. Right-click **CTextView** in ClassView and declare a protected function as follows:

```
void ComputeViewMetrics()
```

Notes

☐ 4. Add public member functions manually to the file CTextView.h as follows:

```
CSize       GetDocSize() const { return m_DocSize; }
CSize       GetCharSize() const { return m_ViewCharSize; }
```

☐ 5. Open the file TextView.cpp.

☐ 6. Initialize m_ViewCharSize, m_DocSize, and m_pFont in the constructor. The constructor looks like the following:

```
CTextView:: CTextView()
    :   m_ViewCharSize(0,0),
        m_DocSize(0,0)
{
    m_pFont = NULL;
}
```

☐ 7. Save TextView.cpp.

▶ **Get the current font**

☐ 1. In TextView.cpp, define the **GetFont** function as follows:

```
CFont*  CTextView::GetFont()
```

☐ 2. If no font has been created, construct a new font as follows:

```
if(m_pFont == NULL)
{
    m_pFont = new Cfont;
```

☐ 3. Create a nine-point Arial font in m_pFont as follows:

```
if(m_pFont)
{
    //  Default to 9 pt Arial
    m_pFont->CreatePointFont(90, "Arial");
}
```

☐ 4. Return m_pFont. Save TextView.cpp.

Notes

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
CFont * CTextView::GetFont()
{

    if(m_pFont == NULL)
    {
        m_pFont = new CFont;
        if(m_pFont)
        {
            //  Default to 9 pt Arial
            m_pFont->CreatePointFont(90, "Arial");
        }
    }
    return m_pFont;
}
```

▶ **Compute the basic metrics for the view by defining ComputeViewMetrics**

☐ 1. Get the pointer to the screen device context (DC) and save the state of the DC as follows:

```
CDC* pDC = CDC::FromHandle(::GetDC(NULL));
    int nSaveDC = pDC->SaveDC();
```

☐ 2. Set the mapping mode to MM_LOENGLISH as follows:

```
pDC->SetMapMode(MM_LOENGLISH);
```

☐ 3. Select the display font into the DC and get the font's text metrics as follows:

```
CFont* pPreviousFont = pDC->SelectObject(GetFont());
TEXTMETRIC tm;
pDC->GetTextMetrics(&tm);
```

☐ 4. Calculate the height of a font element (character) as the sum of its internal height (tmHeight) and the space between lines (tmExternalLeading):

```
m_ViewCharSize.cy = tm.tmHeight + tm.tmExternalLeading;
m_ViewCharSize.cx = tm.tmAveCharWidth;
```

☐ 5. Get a pointer to a document so you can access the **CStringList** member that holds the data:

```
CTextDoc* pDoc = GetDocument();
```

☐ 6. Initialize the document width to 0. To calculate the document height, multiply the number of lines by the height of a line:

```
m_DocSize.cx = 0;
m_DocSize.cy = m_ViewCharSize.cy *
                pDoc->GetLineList()->GetCount();
```

☐ 7. The longest line has to be calculated by looking at each line of the document using the current font. Declare variables for a loop to interrogate each line as follows:

```
CString Line;
CSize size;
```

Notes

☐ 8. Because **CStringList** is a collection with an iterator, you will use a POSITION pointer to iterate through the list:

```
POSITION pos = pDoc->GetLineList()->GetHeadPosition();
while( pos != NULL )
```

☐ 9. Get the current line, and from it get its text extent as follows:

```
Line = pDoc->GetLineList()->GetNext( pos );
size = pDC->GetTextExtent(Line, Line.GetLength());
```

☐ 10. Set the width of the document to the largest size found as follows:

```
m_DocSize.cx = max(size.cx, m_DocSize.cx);
```

☐ 11. After the loop is closed, add a four-pixel margin as follows:

```
m_DocSize.cx += 4 * m_ViewCharSize.cx;
```

☐ 12. Select the application font out of the DC as follows:

```
if(pPreviousFont)
    {
    pDC->SelectObject(pPreviousFont);
    }
```

☐ 13. Restore the DC to its original state as follows:

```
pDC->RestoreDC(nSaveDC);
```

☐ 14. Release the DC as follows:

```
::ReleaseDC(NULL, pDC->GetSafeHdc());
```

☐ 15. Save TextView.cpp and TextView.h.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CTextView::ComputeViewMetrics()
{

    // get a CDC* for the screen
    CDC* pDC = CDC::FromHandle(::GetDC(NULL));
    int nSaveDC = pDC->SaveDC();


    // select mapping mode
    pDC->SetMapMode(MM_LOENGLISH);

    // select the font and get its metrics
    CFont* pPreviousFont = pDC->SelectObject(GetFont());
    TEXTMETRIC tm;
    pDC->GetTextMetrics(&tm);
```

(Code continues on following page.)

Notes

```
    //  Calculate view character size
    m_ViewCharSize.cy = tm.tmHeight + tm.tmExternalLeading;
    m_ViewCharSize.cx = tm.tmAveCharWidth;

    // convert to device units to minimize round off error
    pDC->LPtoDP(&m_ViewCharSize);

    //  Calculate document size
    CTextDoc* pDoc = GetDocument();
    m_DocSize.cy = m_ViewCharSize.cy *
                   pDoc->GetLineList()->GetCount();

    // loop through the document and find the longest line
    CString Line;
    CSize size;
    POSITION pos = pDoc->GetLineList()->GetHeadPosition();
    while( pos != NULL )
    {
        Line = pDoc->GetLineList()->GetNext( pos );
        size = pDC->GetTextExtent(Line, Line.GetLength());
        m_DocSize.cx = max(size.cx, m_DocSize.cx);
    }

    //  Account for our simple margin
    m_DocSize.cx += 4 * m_ViewCharSize.cx;

    // clean up
    if(pPreviousFont)
    {
        pDC->SelectObject(pPreviousFont);
    }
    pDC->RestoreDC(nSaveDC);
    ::ReleaseDC(NULL,pDC->GetSafeHdc());
}
```

▶ **Implement the OnDraw function**

In the **OnDraw** function, you need to calculate the number of lines that can fit into the window and paint only those lines. In addition, you will need to process the OnUpdate message that is sent when the view window is resized.

☐ 1. Move to the top of **CTextView::OnDraw**. Delete the contents of the function and declare variables to store the first and last lines of the visible text:

```
int nFirstLn, nLastLn;
```

☐ 2. Calculate the lines to draw by calling **ComputeVisibleLines** as follows:

```
ComputeVisibleLines(pDC, nFirstLn, nLastLn);
```

Notes

☐ 3. Calculate the position of the first line relative to the origin of the window as follows:

```
int nYPos = - nFirstLn * GetCharSize().cy;
int nXPos = 4 * GetCharSize().cx;
```

☐ 4. Call the core **OnDraw** handler as follows:

```
OnDraw(pDC, nFirstLn, nLastLn,nXPos,nYPos);
```

☐ 5. Save TextView.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CTextView::OnDraw(CDC* pDC)
{
    int nFirstLn, nLastLn;

    ComputeVisibleLines(pDC, nFirstLn, nLastLn);

    int nYPos = - nFirstLn * GetCharSize().cy;
    int nXPos = 4 * GetCharSize().cx;
    OnDraw(pDC, nFirstLn, nLastLn,nXPos,nYPos);
}
```

▶ **Implement the ComputeVisibleLines function**

☐ 1. Right-click **CTextView** in ClassView and add **ComputeVisibleLines** as a protected function:

```
void ComputeVisibleLines(CDC* pDC, int& nFirst, int& nLast)
```

☐ 2. Begin writing the code for the **ComputeVisibleLines** function by getting the number of lines in the **CStringList** as folllows:

```
int nLineCount = GetDocument()->GetLineList()->GetCount();
```

☐ 3. Get the viewport origin in logical coordinates as follows:

```
CPoint pt = pDC->GetViewportOrg();
pDC->DPtoLP(&pt,1);
```

☐ 4. Get the clipping region in logical coordinates as follows:

```
CRect rc;
pDC->GetClipBox(&rc);
```

☐ 5. Get the line height as follows:

```
CSize CharSize = GetCharSize();
```

☐ 6. The algorithm for the first visible line accomplishes these points:

a. Calculates the distance from the top of the viewport to the top of the clipping region.

b. Divides this distance by the height of a line, giving the number of lines.

c. Ensures that at least one line will be shown.

The code is written as follows:

```
nFirst = min(abs((rc.top - pt.y)/CharSize.cy),
           nLineCount-1);
```

Notes

☐ 7. The algorithm for the last visible line accomplishes these points:

a. Calculates the number of lines that will fit into the clipping region.

b. Adds that to the starting line.

c. Adds one more line to make sure that partial lines are displayed.

d. Ensures that this is less than the total number of lines.

The code is written as follows:

```
nLast = min(abs(rc.Height())/CharSize.cy + nFirst + 1,
            nLineCount-1);
```

☐ 8. Save TextView.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CTextView::ComputeVisibleLines(CDC* pDC, int& nFirst, int& nLast)
{
    int nLineCount = GetDocument()->GetLineList()->GetCount();

    // Get the viewport origin, convert to logical coordinates
    CPoint pt = pDC->GetViewportOrg();
    pDC->DPtoLP(&pt,1);

    // Get the clipping region, in logical coordinates
    CRect rc;
    pDC->GetClipBox(&rc);

    // Get the logical line height
    CSize CharSize = GetCharSize();

    // Compute the first visible line
    nFirst = min(abs((rc.top - pt.y)/CharSize.cy),
                 nLineCount-1);

    // Compute the last visible line
    nLast = min(abs(rc.Height())/CharSize.cy + nFirst + 1,
                 nLineCount-1);
}
```

▶ **Implement the core OnDraw handler**

☐ 1. Declare a second **OnDraw** handler in TextView.h as follows:

```
virtual void OnDraw(CDC* pDC, int nFirstLn, int nLastLn,
                    int nXPos = 0, int nYPos = 0);
```

☐ 2. Define the second **OnDraw** handler in TextView.cpp as follows:

```
void CTextView::OnDraw(CDC* pDC, int nFirstLn, int nLastLn,
                       int nXPos /*= 0*/, int nYPos /*= 0*/)
```

Notes

☐ 3. Select your chosen font into the DC as follows:

```
CFont* pPreviousFont = pDC->SelectObject(GetFont());
```

☐ 4. Get the size of the font as follows:

```
CSize CharSize = GetCharSize();
```

☐ 5. Get the string list from the document as follows:

```
CStringList *pLineList = GetDocument()->GetLineList();
```

☐ 6. You will loop through the lines in pLineList from the first line passed (which will be an index) to the last line passed, drawing the text on the screen and moving down the screen (that is, to lower y-coordinate values). Declare the necessary variables:

```
CString     strLine;
POSITION    pos;
```

☐ 7. Control the loop as follows:

```
while (nFirstLn <= nLastLn)
```

☐ 8. Find the item in the list as follows:

```
if( ( pos = pLineList->FindIndex( nFirstLn )) != NULL )
```

☐ 9. If you have a valid item, copy it to the string and display it as follows:

```
strLine = pLineList->GetAt(pos);
pDC->TabbedTextOut(nXPos, nYPos, strLine, 0, NULL, 0);
```

☐ 10. Decrement the y-coordinate and increment the line count as follows:

```
nYPos -= CharSize.cy;
nFirstLn++;
```

☐ 11. Back outside the loop, select your font out of the DC:

```
if(pPreviousFont)
{
    pDC->SelectObject(pPreviousFont);
}
```

☐ 12. Save TextView.cpp.

Notes

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CTextView::OnDraw(CDC* pDC, int nFirstLn, int nLastLn,
                       int nXPos /*= 0*/, int nYPos /*= 0*/)
{
    //  Select specified font
    CFont* pPreviousFont = pDC->SelectObject(GetFont());

    //  Needed for height of each line
    CSize CharSize = GetCharSize();

    //  Get list of strings from the document
    //  and output them to the display context
    CStringList *pLineList = GetDocument()->GetLineList();

    CString     strLine;
    POSITION    pos;
    while (nFirstLn <= nLastLn)
    {
        if( ( pos = pLineList->FindIndex( nFirstLn )) != NULL )
        {
            strLine = pLineList->GetAt(pos);
            pDC->TabbedTextOut(nXPos, nYPos, strLine, 0, NULL, 0);
            nYPos -= CharSize.cy;
            nFirstLn++;
        }
    }

    //  Cleanup and restore original GDI Objects
    if(pPreviousFont)
    {
        pDC->SelectObject(pPreviousFont);
    }
}
```

▶ **Implement OnUpdate**

☐ 1. Using ClassWizard, delete **CTextView::OnInitialUpdate** and manually remove the associated code.

☐ 2. Using ClassWizard or WizardBar, add **CTextView::OnUpdate**. Edit the code to compute the view metrics as follows:

```
ComputeViewMetrics();
```

☐ 3. **CScrollView::SetScrollSizes** sets the mapping mode for the scroll view. Because **CTextView::ComputeViewMetrics** uses MM_LOENGLISH for its calculations, you will need to use that mode here. It also sets the scrolling ranges. Get these ranges from the document size:

```
SetScrollSizes( MM_LOENGLISH, GetDocSize());
```

Notes

☐ 4. **CView::OnUpdate** is sent whenever the view window is changed. Invalidate the window so that **CTextView::OnDraw** will be called to appropriately redisplay the text:

```
Invalidate();
```

☐ 5. Save TextView.cpp.

To see an example of how your code should look, click this icon.

(CD-ROM displays the following sample code.)

```
void CTextView::OnUpdate(CView* pSender, LPARAM lHint, CObject* pHint)
{
    ComputeViewMetrics();

    SetScrollSizes(MM_LOENGLISH, GetDocSize());
    Invalidate();
}
```

☐ 6. Save TextView.h and TextView.cpp.

☐ 7. Build your application.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.3\Ex01\Solution.

Notes

123 Exercise 2: Adding Font Support

Continue with the files you created in Exercise 1 or, if you do not have a starting point for this exercise, the code that forms the basis for this exercise is in \Labs\Ch04\Lab4.3\Ex02.

In this exercise, you will implement a user interface to **CTextView.m_pFont**.

▶ Add a menu item for Font

☐ 1. Open the IDR_TEXTTYPE menu resource.

☐ 2. Add a menu between the **View** and **Window** menus. Give it the caption **Font**.

☐ 3. Add a menu item below the font menu, giving it the ID **ID_FORMAT_FONT** and the prompt, **Change Font**.

☐ 4. Save the file Text.rc.

▶ Add a handler for the menu message

- Use ClassWizard or WizardBar. Add a handler for ID_FORMAT_FONT to **CTextView**, and accept the default **OnFormatFont** function name.

▶ Implement CTextView::OnFormatFont

☐ 1. Go to the head of **OnFormatFont**. Get (or create, if it has not yet been created) the current font with **GetFont** as follows:

```
CFont * pFont = GetFont();
```

☐ 2. Retrieve a LOGFONT structure with the font information as follows:

```
LOGFONT lf;
pFont->GetObject(sizeof(LOGFONT), &lf);
```

☐ 3. Use this structure to initialize a common font dialog box as follows:

```
CFontDialog dlg(&lf, CF_SCREENFONTS | CF_INITTOLOGFONTSTRUCT);
```

☐ 4. Show the dialog box modally:

```
if(dlg.DoModal() == IDOK)
```

☐ 5. If the user closed the font dialog box with **OK**, delete the current font (remember that this font is selected out of a device context (DC) each time it is selected in, so this deletion is safe):

```
if(m_pFont)
{
    delete m_pFont;
}
```

☐ 6. Construct the new font and initialize it using the LOGFONT structure returned from **CFontDialog** as follows:

```
m_pFont = new CFont;
if(m_pFont)
{
    m_pFont->CreateFontIndirect(&lf);
}
```

Notes

☐ 7. Finally, you must recalculate all the metrics and invalidate the view's window to redisplay the file using the new font. You already have a function that does this, **OnUpdate**, but it will take some significant amount code to set up the call. The simplest way to accomplish this is to call **CDocument::UpdateAllViews**. Because there is only one view of this document, this will be the equivalent of calling **OnUpdate** directly:

```
GetDocument()->UpdateAllViews(NULL);
```

☐ 8. Save TextView.cpp.

To see an example of how your code should look, click this icon.
(CD-ROM displays the following sample code.)

```
void CTextView::OnFormatFont()
{

    CFont * pFont = GetFont();

    LOGFONT lf;
    pFont->GetObject(sizeof(LOGFONT), &lf);

    CFontDialog dlg(&lf, CF_SCREENFONTS | CF_INITTOLOGFONTSTRUCT);

    if(dlg.DoModal() == IDOK)
    {
        if(m_pFont)
        {
            delete m_pFont;
        }

        m_pFont = new CFont;
        if(m_pFont)
        {
            m_pFont->CreateFontIndirect(&lf);
        }

        //  This will cause OnUpdate() to be called ensuring
        //  that our cached metrics and scrolling get updated
        GetDocument()->UpdateAllViews(NULL);
    }
}
```

▶ **Build and test your application**

- Run your application and open a file. The view class supports scrolling and changes of fonts. Select the **font** option to change the text font.

The solution code for this exercise is located in the folder \Labs\Ch04\Lab4.3\Ex02\Solution.

Notes